

Arrays

- ✚ An **array** is a collection of a fixed number of components (also called elements) all of the same data type and in contiguous (that is, adjacent) memory space.

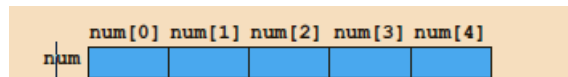
7-1 A one-dimensional array

It is an array in which the components are arranged in a list form. The general form for declaring a one-dimensional array is:

```
dataType arrayName(intExp);
```

- ✚ in which **intExp** specifies the number of components in the array and can be any constant expression that evaluates to a positive integer.
- ✚ For example: The statement: **int num[5];**

declares an array num of five components. Each component is of type int. The component names are num[0], num[1], num[2], num[3], and num[4]. Figure bellow illustrates the array num.



- ✚ **For accessing Array Components,**
The general form (syntax) used for accessing an array component is:
arrayName[indexExp]

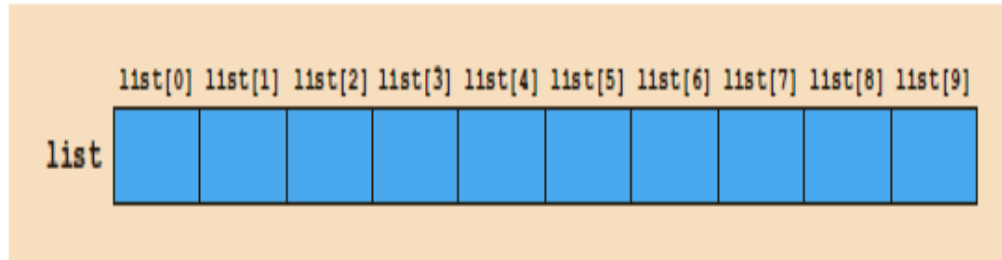
in which **indexExp**, called the **index**, is any expression whose value is a nonnegative integer.

- ✚ The index value specifies the position of the component in the array. In C++, [] is an operator called the **array subscripting operator**. Moreover, in C++, the array index starts at **0**.

Consider the following statement:

int list[10];

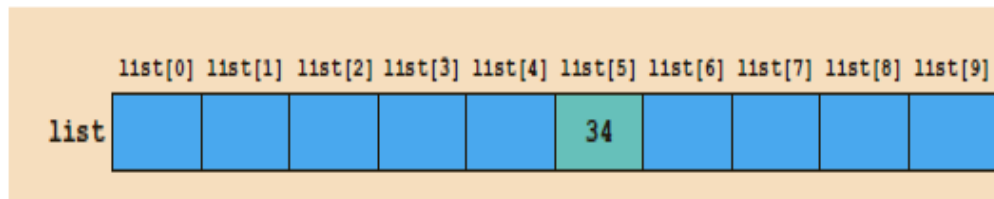
This statement declares an array **list** of **10** components. The components are **list[0]**, **list[1]**, . . . , **list[9]**. In other words, we have declared 10 variables (see Figure below).



✚ The assignment statement:

list[5] = 34;

stores 34 in list[5], which is the sixth component of the array list (see Figure below).



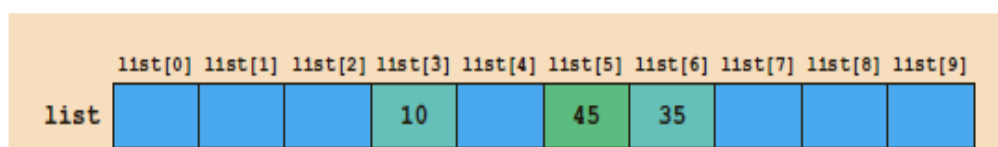
✚ Consider the following statements:

list[3] = 10;

list[6] = 35;

list[5] = list[3] + list[6];

The first statement stores 10 in list[3], the second statement stores 35 in list[6], and the third statement adds the contents of list[3] and list[6] and stores the result in list[5] (see Figure below).



7-2 Processing One-Dimensional Arrays

Some of the basic operations performed on a one-dimensional array are initializing, inputting data, outputting data stored in an array, and finding the largest and/or smallest element. Moreover, if the data is numeric, some other basic operations are finding the sum and average of the elements of the array. Each of these operations requires the ability to step through the elements of the array. This is easily accomplished using a loop. For example, suppose that we have the following statements:

```
int list[100]; //list is an array of size 100
int i;
```

The following **for** loop steps through each element of the array **list**, starting at the first element of **list**:

```
for (i = 0; i < 100; i++)    //Line1 process list[i]
```

If processing the list requires inputting data into **list**, the statement in Line 2 takes the form of an input statement, such as the **cin** statement. For example, the following statements read 100 numbers from the keyboard and store the numbers in **list**:

```
for (i = 0; i < 100; i++)
cin >> list[i];
```

Similarly, if processing **list** requires outputting the data, we write the following code.

```
for (i = 0; i < 100; i++) {
cout << list[i] << " ";
cout << endl;}
```

Example (1):

For array of sales read its elements and find **the summation, average, and large Sale value, and its position.**

Sol.

```
#include <iostream>
using namespace std;
int main() {

// declaration
double sales[10];
double largestSale, sum, average;

// Initializing an array: The following loop initializes every component of the array sales to 0.0.
for (int index = 0; index < 10; index++)
{
    sales[index] = 0.0;
}

// Reading data into an array
for (int index = 0; index < 10; index++)
{
    cin >> sales[index];
}

//Printing an array
cout <<"the sales are : "<< endl;
for (int index = 0; index < 10; index++)
{
    cout<<sales[index] << " ";
}
cout<<"\n";

//Finding the sum and average of an array
sum = 0;
for (int index = 0; index < 10; index++)
{
    sum = sum + sales[index];
    average = sum / 10;
}
```

```

}
cout <<"summation of sales is: " << sum<< endl;
cout <<"average of sales is: " << average<< endl;

// finding the Largest element in the array
int maxIndex = 0;
for (int index = 1; index < 10; index++)
{
    if (sales[maxIndex] < sales[index])
        maxIndex = index;
    largestSale = sales[maxIndex];
}
cout <<"largestSale of sales is: " << largestSale;

return 0;
}

```

Output:

```

1 2 3 4 10 5 6 7 8 9
the sales are :
1 2 3 4 10 5 6 7 8 9
summation of sales is: 55
average of sales is: 5.5
largestSale of sales is: 10
the position of the largeSale is:4

```

Example 2:-

Write C++ program, to split the odd numbers and even numbers of one array into two arrays?

Sol.

```

#include <iostream>
using namespace std;
int main() {
    int a [20]= { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 };
    int aodd[20], aeven [20];
    int i ,o=0, e=0;
    for ( i=0 ; i<20; i++ )
    {
        if (a[i] % 2 !=0)

```

```
{
aodd[o]=a[i];
o=o+1;
}
else
{
aeven[e]=a[i];
e=e+1;
}
}

cout<<"the array of odd number is : ";
for ( i=0 ; i<o; i++ )
cout<<aodd[i]<<" ";
cout<<endl;

cout<<"the array of even numbers is :";
for ( i=0 ; i<e; i++ )
cout<<aeven[i]<<" ";
return 0;
}
```

output

the array of odd number is : 1 3 5 7 9 11 13 15 17 19

the array of even numbers is :2 4 6 8 10 12 14 16 18 20

Example 3:

Write a program in C++ language to obtain and print the sum of elements of array, which values equal to square index of array one-dimension size 5.

Sol.

```
#include <iostream>
using namespace std;
int main() {
    int a[5];
    int sum =0;
    for (int i=0;i<5;++i){
        a[i]=i*i;
        cout <<"a["<<i <<"]="<<a[i]<<"\t";
        sum +=a[i];
    }
    cout <<"\nsum ="<<sum <<endl;
    return 0;
}
```